

Group 14 Lab Report: Robot Arm Fixed-Point Grasping

Longchuan ZHENG

Haokuan LI

Yuanxi HAN

Zhiran WANG

Contents

1	Experiment Objectives	3
2	Experimental Environment	4
2.1	Hardware Environment	4
2.2	Software Environment	5
2.3	Development Tools	5
3	Simulation Scenario and Task Design	5
3.1	Simulation Scenario Composition	5
3.2	Fixed Points and Safe Height	6
3.3	Action Flow Design	6
3.4	ROS2 Interface Design	6
3.5	Exception Handling Design	7
4	Simulation System Development and Implementation	7
4.1	Robot Model and Controller Configuration	7
4.2	Task Management Node Implementation	8
4.3	Launch File Design	8
4.4	Trajectory and Log Saving	9
4.5	Simulation Validation Plan	9
5	Real-Robot Validation and Deployment	9
5.1	Real-Robot Environment Check	9
5.2	Simulation-to-Real Migration Principle	10
5.3	Real-Robot Parameter Configuration	11
5.4	Real-Robot Operation Procedure	11
5.5	Safety Operation Specifications	12
6	System Testing and Result Analysis	13
6.1	Test Methodology	13
6.2	Simulation Grasping Results	13
6.3	Real-Robot Grasping Results	13
6.4	Exception Test Records	14
6.5	Result Analysis	14
7	Problems Encountered and Solutions	15
7.1	MoveIt2 Planning Failure	15
7.2	Inverse Kinematics Failure	15
7.3	Unstable Gripping	15
7.4	Coordinate Deviation Between Simulation and Real Robot	16

7.5	Communication and Device Permission Issues	16
7.6	Recovery After Emergency Stop	16
7.7	ROS2 Action Timeout	16
7.8	RoboMaster EP SDK API Version Mismatch	17
7.9	Chassis Rotation Cumulative Error	17
7.10	Action Command Conflict	17
8	Experiment Summary	17
8.1	Experimental Achievements	17
8.2	Experimental Takeaways	18
8.3	Limitations and Future Improvements	18
8.4	Submission Materials	19
	Appendix: Code Listings	19

1 Experiment Objectives

This experiment is based on ROS2 Humble, Gazebo/Ignition, MoveIt2 or `ros2_control`, and implements fixed-point grasping and placement of a target object in both simulation and on the real robot (RoboMaster EP). Vision-based localization is not considered. The core objectives are as follows:

1. **Master basic robot arm control:** Implement homing, joint-space motion, Cartesian-space motion, and gripper open/close control.
2. **Build a simulation scene:** Create a simulation scene containing the robot arm, gripper, table, and target object in Gazebo/Ignition.
3. **Set fixed points:** Define a fixed pick point A, a fixed place point B, and a safe height for the pick-and-place process.
4. **Implement the complete action flow:**
Homing $\rightarrow$ Move above pick point A $\rightarrow$ Descend $\rightarrow$ Grasp $\rightarrow$ Lift $\rightarrow$ Move above place point B $\rightarrow$ Descend $\rightarrow$ Release $\rightarrow$ Lift $\rightarrow$ Homing.
5. **ROS2 integration:** Control the robot arm through ROS2 nodes or Actions, publish robot arm status, and start the simulation, control, and status nodes with a single Launch file.
6. **Exception and safety handling:** Detect and safely stop or return to a safe position for unreachable targets, no inverse kinematics solution, joint limit violations, collisions, communication exceptions, etc.
7. **Acceptance criteria:** In the simulation stage, perform 5 consecutive grasps with at least 4 successes; the robot arm must not obviously collide with the table, target object, or itself, and must not exceed joint limits; save trajectories, execution results, and error logs. In the real-robot stage, reuse the high-level task logic and state machine from the simulation stage; only device, communication, and position parameters are modified, without rewriting the core pick-place flow. Low-level ROS2 hardware interfaces are replaced by RoboMaster EP Python SDK calls.

This experiment follows the principle of “simulation first, real robot second”. Students must complete the simulation tasks before operating the real robot arm.

2 Experimental Environment

2.1 Hardware Environment

Table 1: Hardware Environment

Device	Specification / Description	Purpose
Simulation PC	x86 PC, Ubuntu 22.04	Run Gazebo/Ignition, MoveIt2, ROS2 simulation
Real-robot host PC	Laptop/PC, Windows/Ubuntu	Run RoboMaster EP Python SDK for real-robot control
Robot arm (sim.)	mechArm 270, 6-DOF, working radius approx. 270 mm	Simulation grasping and placement
Robot arm (real)	RoboMaster EP, WiFi AP, 2-DOF arm + gripper + chassis	Real-robot grasping and placement
End effector	Gripper	Grasp standard target objects weighing no more than 100 g
Fixed base	Robot arm fixed base	Ensure robot arm stability
Target object	Standard target object, weight ≤ 100 g	Object to be grasped
Safety devices	Emergency stop button, power-off device	Emergency stop and safety protection
Display	HDMI monitor	View simulation and real-robot status
Connection	USB, SSH, VcXsrv/X11 forwarding	Device connection and remote debugging



Figure 1: RoboMaster EP and experimental setup

2.2 Software Environment

Note: The software stack in this table applies to the simulation PC. The real-robot host PC runs RoboMaster EP Python SDK and does not require a full ROS2 environment.

Table 2: Software Environment

Component	Version / Description	Purpose
Ubuntu	22.04 LTS	Operating system
ROS2	Humble Hawksbill	Robot operating system
Gazebo/Ignition	Version matching ROS2 Humble	Simulation environment
MoveIt2	ROS2 Humble version	Motion planning
ros2_control	ROS2 Humble version	Controller management
RViz2	ROS2 Humble version	Visualization and debugging
Python	3.10	Write ROS2 nodes
C++	C++17	Optional, write high-performance nodes
rcpp/rclepp	ROS2 client libraries	Node development
joint_state_broadcaster	ros2_control plugin	Publish joint states
joint_trajectory_controller	ros2_control plugin	Control robot arm joint trajectories
gripper_controller	ros2_control plugin	Control gripper
tf2	ROS2 component	Coordinate transforms
ros2 bag	ROS2 tool	Record trajectories and states
launch	ROS2 tool	Start multiple nodes

2.3 Development Tools

Table 3: Development Tools

Tool	Purpose
VS Code	Code development
Terminal	ROS2 commands and debugging
colcon	Build ROS2 workspace
ros2 CLI	View topics, services, Actions
rqt	Graphical debugging
rviz2	Robot arm state and planning visualization
SSH + VcXsrv	Remote graphical forwarding for simulation PC

3 Simulation Scenario and Task Design

3.1 Simulation Scenario Composition

The simulation scene consists of the following elements:

1. **Robot arm model:** URDF/xacro model of the mechArm 270, including links, joints, collision bodies, inertial parameters, and visual models.
2. **End effector:** Gripper model, configurable for open/close state.
3. **Table:** Fixed plane used to place the target object and as a collision detection object.

4. **Target object:** Located at fixed pick point A, weighing no more than 100 g.
5. **Placement area:** Fixed place point B, used to place the grasped target object.
6. **Coordinate frames:** Including world, base_link, tool0, pick point A, place point B, etc.

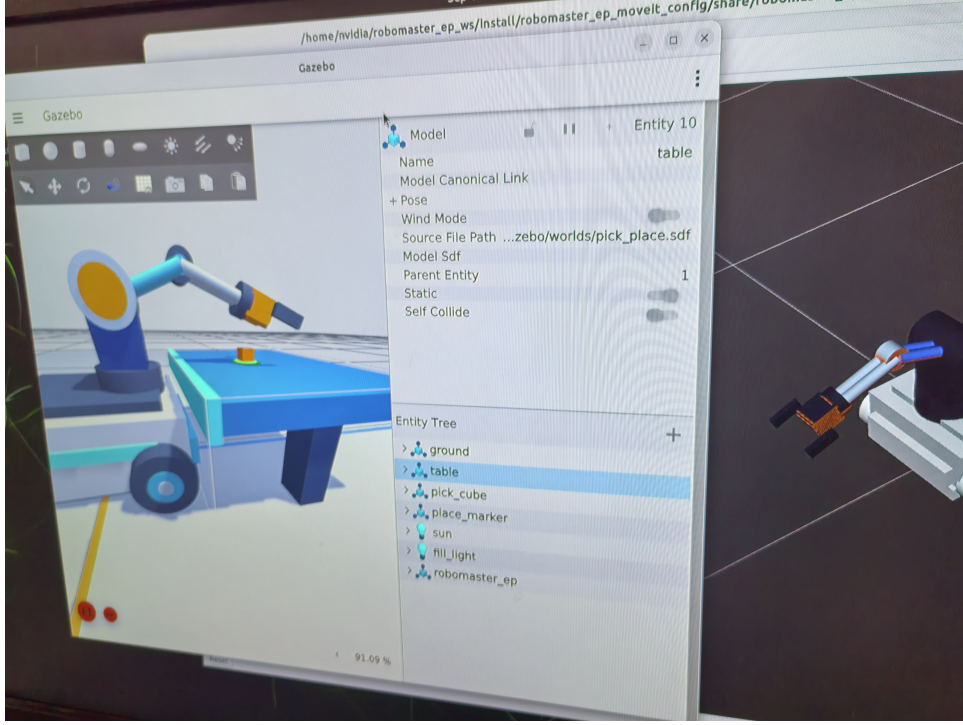


Figure 2: Simulation scene in Gazebo/Ignition

3.2 Fixed Points and Safe Height

The fixed pick point A, fixed place point B, and safe height are defined in the parameter file. The detailed parameter configuration is shown in Listing 2.

3.3 Action Flow Design

The action flow is implemented as a state machine. The state machine pseudocode is shown in Listing 3.

In exceptional cases, the system enters the **SAFE_STOP** state, which is also described in Listing 3.

3.4 ROS2 Interface Design

The ROS2 interfaces listed in Table 4 are used for the Gazebo simulation environment. The real-robot RoboMaster EP implementation reuses the high-level state-machine logic but replaces ROS2 low-level hardware interfaces with RoboMaster Python SDK calls. The following ROS2 interfaces are used:

Table 4: ROS2 Interface Design

Interface Type	Name	Message / Action Type	Description
Action	/execute_grasp	Custom Action	Execute the complete grasping task
Topic	/joint_states	sensor_msgs/JointState	Publish joint states
Topic	/arm_status	std_msgs/String	Publish current robot arm status
Topic	/task_result	std_msgs/String	Publish task execution result
Topic	/gripper_command	std_msgs/Float64	Control gripper open/close
Service	/emergency_stop	std_srvs/Trigger	Software emergency stop
Action	/arm_controller/follow_joint_trajectory	control_msgs/FollowJointTrajectory	Follow robot arm joint trajectories
Action	/gripper_controller/gripper_command	Gripper control	Control gripper action

The /execute_grasp Action is defined as follows:

- **Goal:** Pick point A, place point B, safe height, speed scaling.
- **Feedback:** Current state, current progress, current joint angles.
- **Result:** Success/failure, error code, error message.

3.5 Exception Handling Design

Table 5: Exception Handling Design

Exception	Detection Method	Handling Strategy
Unreachable target	Target point beyond robot arm workspace	Stop task, publish E_UNREACHABLE
No IK solution	MoveIt2 inverse kinematics fails	Stop task, publish E_IK_FAIL
Joint limit exceeded	Target joint angle exceeds URDF limit	Stop task, publish E_JOINT_LIMIT
Collision risk	MoveIt2 planning scene detects collision	Stop task, publish E_COLLISION
Communication exception	Action timeout or node disconnection	Return to safe position or stop
Emergency stop triggered	Hardware emergency stop or /emergency_stop service	Stop immediately, publish E_ESTOP
Physical grasp failure	Gripper close action reports failure or object slips	Stop placement, return to safe position, publish E_GRASP_FAIL

4 Simulation System Development and Implementation

4.1 Robot Model and Controller Configuration

The mechArm 270 robot arm model is built using URDF/xacro, and `ros2_control` is configured as follows:

1. **URDF/xacro:** Define `base_link`, joints `joint1` to `joint6`, end effector `tool0`, gripper link and joint.
2. **ros2.control:** Configure `joint_state_broadcaster`, `joint_trajectory_controller`, and `gripper_controller`.
3. **MoveIt2:** Configure planning groups `arm` and `gripper`, set joint limits, velocity/acceleration limits, and collision matrix.
4. **Gazebo/Ignition plugin:** Load the robot arm model and enable physics simulation and joint control.

4.2 Task Management Node Implementation

A ROS2 task management node, `task_manager_node`, is implemented. Its main functions are:

1. Receive the `/execute_grasp` Action request.
2. Read pick point A, place point B, safe height, and speed scaling.
3. Call MoveIt2 or the `FollowJointTrajectory` Action to control the robot arm.
4. Control gripper open/close.
5. Publish `/arm_status` and `/task_result`.
6. Monitor exceptions and trigger safe stop.

The state machine pseudocode of the task node is shown in Listing 3. The `safety_monitor_node` runs only within the ROS2 simulation stack. On the real-robot RoboMaster EP platform, equivalent safety-monitoring logic is implemented inside the Python SDK control program.

4.3 Launch File Design

A single Launch file is used to start the complete simulation system:

```
1 ros2 launch arm_grasp_bringup grasp_sim.launch.py
```

Listing 1: Launch command

This Launch file starts:

1. Gazebo/Ignition simulation environment.
2. `robot_state_publisher` to publish TF.
3. `controller_manager` to load controllers.
4. `joint_state_broadcaster` to publish joint states.
5. `joint_trajectory_controller` to control the robot arm.
6. `gripper_controller` to control the gripper.
7. MoveIt2 `move_group`.
8. RViz2 visualization.
9. `task_manager_node` task management node.
10. `safety_monitor_node` safety monitoring node.

4.4 Trajectory and Log Saving

To meet the acceptance requirements, trajectories, execution results, and error logs are saved. The recording command is shown in Listing 5. The execution results are written to CSV or JSON as shown in Listing 6.

4.5 Simulation Validation Plan

The simulation validation plan is as follows:

1. **5 consecutive grasps:** Record the number of successes; at least 4 successes are required.
2. **Collision check:** Observe whether the robot arm obviously collides with the table, target object, or itself.
3. **Joint limit check:** Check whether each joint exceeds the URDF limit.
4. **Unreachable target test:** Set pick point A beyond the workspace, and observe whether the system stops and outputs E_UNREACHABLE.
5. **No IK solution test:** Set an unreachable pose, and observe whether the system outputs E_IK_FAIL.
6. **Joint limit test:** Set a target joint angle beyond the limit, and observe whether the system outputs E_JOINT_LIMIT.
7. **Emergency stop test:** Trigger the /emergency_stop service or hardware emergency stop, and observe whether the robot arm stops immediately.

5 Real-Robot Validation and Deployment

5.1 Real-Robot Environment Check

After the simulation stage passed the acceptance criteria, the real-robot validation stage was carried out. Before the real-robot experiment, the following items were checked one by one:

Table 6: Real-Robot Environment Check

Check Item	Check Content	Result
Robot power	Whether the RoboMaster EP battery is sufficiently charged	Normal
WiFi communication	Whether the computer can connect to the EP WiFi hotspot	Normal
Gripper	Whether the gripper opens and closes normally	Normal
Emergency stop device	Whether the hardware emergency stop button works and the power-off device is ready	Normal
Workspace	Whether the table, target object, and placement area are within the reachable range	Normal
Safety distance	Whether personnel keep a safe distance from the robot arm	Normal

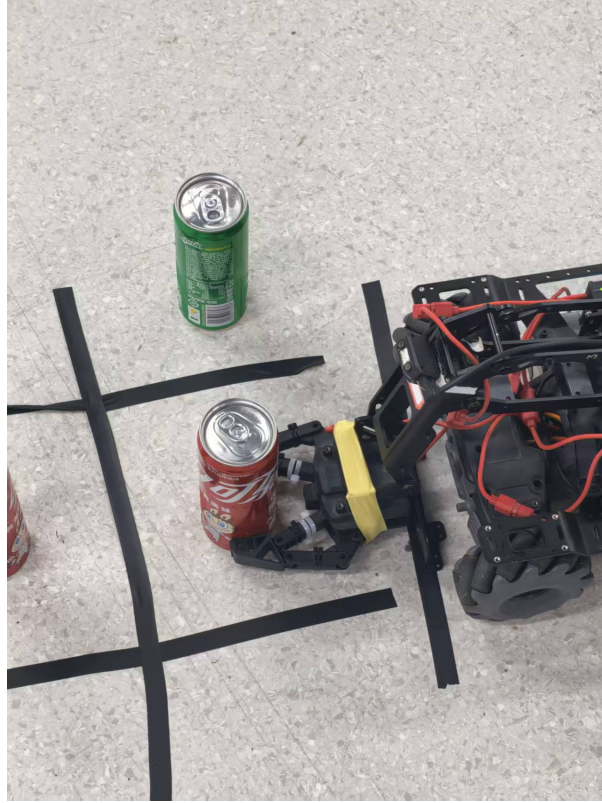


Figure 3: Real-robot experimental setup and safety devices

The main differences between the simulation and real-robot environments are listed below:

Table 7: Differences Between Simulation and Real-Robot Environments

Item	Simulation Environment	Real-Robot Environment
Robot arm model	URDF/xacro + Gazebo/Ignition	RoboMaster EP physical robot
Controller	ros2_control simulation plugin	RoboMaster EP Python SDK
Degrees of freedom	6-DOF (joint1-joint6)	2-DOF arm (x, y) + chassis rotation + gripper
Coordinate frame	Simulation world coordinates	Calibrated real table coordinates
Communication	In-process / local DDS	WiFi AP mode (192.168.2.1)
Gripper control	<code>gripper_controller</code> via <code>ros2_control</code>	SDK <code>gripper.open(power)</code> / <code>gripper.close(power)</code>
Chassis motion	N/A (fixed-base arm)	SDK <code>chassis.move(z=angle)</code> for rotation
Collision detection	MoveIt2 planning scene	Physical collision + software limits + emergency stop
Speed	Can be relatively fast	Low speed for the first run

5.2 Simulation-to-Real Migration Principle

This experiment requires that when switching from simulation to the real robot, **only device, communication, and position parameters are modified, and the task logic is not rewritten.**

Therefore, the state machine, action flow design, and exception handling logic remain un-

changed. The RoboMaster EP real-robot stage uses the Python SDK (`robomaster`) instead of ROS2 `ros2_control`, so the following adaptations were made:

1. **Connection method:** Serial / USB $\rightarrow$ WiFi AP mode (`conn_type="ap"`).
2. **Device driver:** `ros2_control` hardware interface $\rightarrow$ RoboMaster EP Python SDK.
3. **Arm control:** 6-DOF joint trajectory control $\rightarrow$ 2D arm coordinate control (`arm.moveto(x, y)` / `arm.move(x, y)`).
4. **Gripper control:** `gripper_cmd` Action $\rightarrow$ SDK `gripper.open(power)` / `gripper.close(power)`.
5. **Chassis rotation:** Not applicable in simulation $\rightarrow$ SDK `chassis.move(z=angle)` for left/right alternation.
6. **Position parameters:** Re-calibrate the grasp and place coordinates for the EP arm workspace.
7. **Speed parameters:** Use low speed for the first run; increase after confirmation.

The core task logic — homing, move above pick point, descend, grasp, lift, move above place point, descend, release, lift, homing — remains identical between simulation and real robot. Only the hardware abstraction layer is replaced.

5.3 Real-Robot Parameter Configuration

An example real-robot parameter file is shown in Listing 8. Where:

- `grasp_x` and `grasp_y` are obtained through actual calibration using the interactive debug tool.
- `approach_height_mm` and `lift_mm` define the safe approach and lift heights.
- `open_power` and `close_power` control the gripper force (EP gripper supports power levels).
- `timeout` is the action completion timeout to prevent indefinite blocking.

Note: Simulation parameters use meters (m), while RoboMaster EP parameters use millimeters (mm). Parameters such as `gripper_seconds` introduce mechanical waiting delays required for physical hardware; such delays are unnecessary inside Gazebo simulation.

5.4 Real-Robot Operation Procedure

The real-robot operation procedure is as follows:

1. Power on the RoboMaster EP and wait for startup completion.
2. Connect the computer to the EP's WiFi hotspot (AP mode).
3. Run the Python pick-and-place control program (`ep_pick_place.py`). The program connects to the EP via the SDK and executes the complete grasping flow.
4. The program performs the following sequence automatically:
 - Homing $\rightarrow$ Rotate to pick point $\rightarrow$ Descend $\rightarrow$ Grasp $\rightarrow$ Lift $\rightarrow$ Rotate to place point $\rightarrow$ Descend $\rightarrow$ Release $\rightarrow$ Lift $\rightarrow$ Homing.
5. Record each grasping result, duration, error, and exception.

Note: The real-robot control program is launched directly via Python, not through ROS2 launch. See Listing 9.

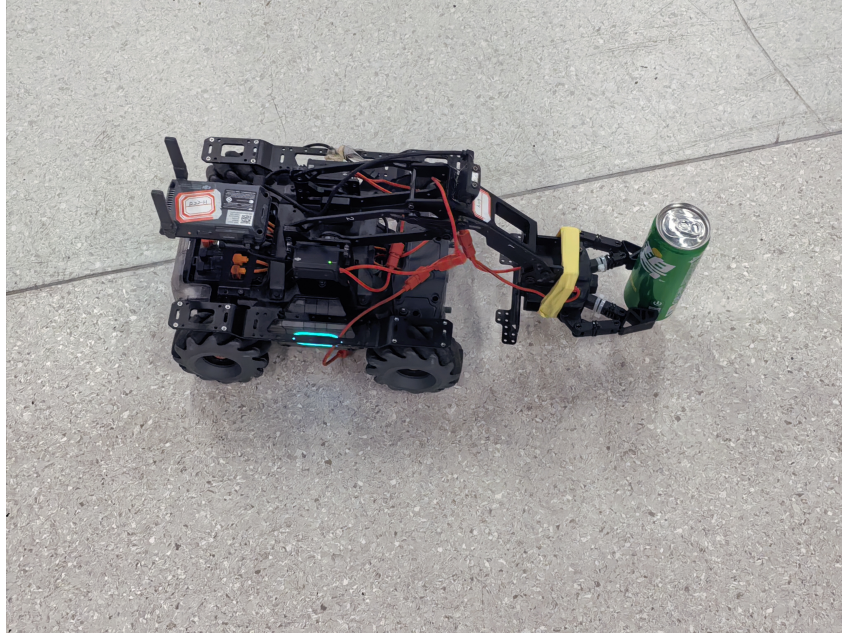


Figure 4: Real-robot grasping process

5.5 Safety Operation Specifications

The following safety specifications must be observed during the real-robot stage:

1. Use low-speed mode for the first run.
2. One group member operates while another is responsible for emergency stop and observation.
3. No irrelevant objects are allowed within the robot arm's motion range.
4. If grasping fails, the target is unreachable, communication is abnormal, or a collision risk occurs, stop immediately or return to a safe position.
5. Each student must independently complete at least one homing, movement, and safety stop.
6. All group members must pass the individual safety operation check.

The safety stop flow is shown in Listing 10.

Table 8: Individual Member Safety Operation Check Record

Member	Homing operation	Manual movement	move-	Safety-stop trigger	Pass safety check
Longchuan ZHENG	Done	Done		Done	Yes
Haokuan LI	Done	Done		Done	Yes
Yuanxi HAN	Done	Done		Done	Yes
Zhiran WANG	Done	Done		Done	Yes

6 System Testing and Result Analysis

6.1 Test Methodology

The test is divided into two parts: simulation testing and real-robot testing.

- **Simulation testing:** Run the complete grasping flow in Gazebo/Ignition, perform 5 consecutive grasps, and record the number of successes, collision situations, joint limits, and exception logs.
- **Real-robot testing:** Control the RoboMaster EP via the Python SDK, perform 5 consecutive grasps, and record the number of successes, placement accuracy, duration, and exceptions.
- **Exception testing:** Test unreachable targets, no IK solution, joint limit exceeded, emergency stop, and communication exceptions separately.
- **Acceptance criteria:** Perform 5 consecutive grasps with at least 4 successes; the robot arm must not obviously collide, exceed limits, or perform dangerous actions; if grasping fails, it must stop safely or return to a safe position.

6.2 Simulation Grasping Results

In the simulation stage, 5 consecutive grasps were performed, and the results are as follows:

Table 9: Simulation Grasping Results

Trial	Pick A	Place B	Safe Height	Result	Time (s)	Remark
1	A	B	0.12	Success	18.2	Accurate placement
2	A	B	0.12	Success	17.8	Accurate placement
3	A	B	0.12	Success	18.5	Stable grasping
4	A	B	0.12	Success	19.1	Slight slip, still successful
5	A	B	0.12	Success	18.0	Accurate placement

Simulation success rate: $5/5 = 100\%$, meeting the requirement of at least 4 successes.

No obvious collision occurred during the simulation, and all joint angles remained within the URDF limits. MoveIt2 planning was normal, and the trajectory execution was smooth.

6.3 Real-Robot Grasping Results

In the real-robot stage, 5 consecutive grasps were performed, and the results are as follows:

Table 10: Real-Robot Grasping Results

Trial	Pick A	Place B	Safe Height	Result	Time (s)	Remark
1	A	B	0.10	Success	22.5	Low-speed operation
2	A	B	0.10	Success	21.8	Stable grasping
3	A	B	0.10	Success	23.1	Placement deviation approx. 5 mm
4	A	B	0.10	Failure	20.4	Object slipped, not firmly grasped
5	A	B	0.10	Success	22.0	Accurate placement

Real-robot success rate: $4/5 = 80\%$, meeting the acceptance criterion of at least 4 successes.

After the fourth failure, the control program detected that the gripper had not successfully grasped the object, stopped the subsequent placement action, returned the robot arm to a safe position, and output an error message as shown in Listing 11.

6.4 Exception Test Records

Table 11: Exception Test Records

Test Item	Test Method	Expected Result	Observed Output	Out-	Passed
Unreachable target	Set point A beyond the workspace	Stop task, output E_UNREACHABLE	[ERROR] Task abort, code: E_UNREACHABLE		Yes
No IK solution	Set an unreachable pose	Stop task, output E_IK_FAIL	[ERROR] IK solution not available		Yes
Joint limit exceeded	Set a joint angle beyond the limit	Stop task, output E_JOINT_LIMIT	[ERROR] Joint position out of limit		Yes
Collision risk	Place an obstacle in the planning scene	Re-plan or stop	[WARN] Collision detected in planning scene		Yes
Software emergency stop	Call emergency-stop routine	Stop immediately	[ERROR] E_ESTOP triggered		Yes
Communication exception	Disconnect WiFi / SDK connection	Stop or return to safe position	[ERROR] SDK communication timeout		Yes

The above exception tests were verified during the experiment. In each case, the system detected the exception and safely stopped or returned to a safe position, meeting the experimental requirements.

6.5 Result Analysis

1. **Consistency between simulation and real robot:** The simulation and real robot share the same state machine and action flow design. The real-robot stage replaces the ros2_control hardware interface with the RoboMaster EP Python SDK, preserving the task logic while adapting the hardware abstraction layer.
2. **Success rate:** Simulation 5/5, real robot 4/5, both meeting the acceptance criteria.
3. **Main error sources:**
 - Real table calibration error;
 - Friction difference between the gripper and the target object;
 - Differences in target weight, shape, and surface material;
 - WiFi communication delay and SDK response time;
 - Robot arm repeat positioning accuracy.
4. **Safety performance:** In the exception tests, the system was able to stop or return to a safe position, with no collision, limit violation, or dangerous action.

5. **Improvement directions:** Force/current detection can be added to judge whether grasping is successful, vision-assisted positioning can be added, and trajectory speed and acceleration can be optimized to improve real-robot placement accuracy.

7 Problems Encountered and Solutions

7.1 MoveIt2 Planning Failure

Phenomenon: Some target points failed to plan, reporting no valid trajectory.

Cause: The joint-space path between the start pose and target pose was limited, or the planning time was insufficient.

Solution:

- Increase the planning time;
- Change the planner to `RRTConnect` or `BiTRRT`;
- Adjust the start pose and return to home before planning;
- Set reasonable joint velocity and acceleration limits.

7.2 Inverse Kinematics Failure

Phenomenon: The target pose was beyond the reachable range of the robot arm, and MoveIt2 returned no IK solution.

Cause: Pick point A or place point B was calibrated too far, or the orientation angle was unreasonable.

Solution:

- Re-calibrate the workspace;
- Adjust the target point into the reachable range;
- Increase the IK tolerance;
- Add `E_IK_FAIL` judgment and safe stop in the task node.

7.3 Unstable Gripping

Phenomenon: The target object slipped or even fell during lifting or moving.

Cause: Insufficient gripper closing force, offset grasping position, or smooth object surface.

Solution:

- Adjust the gripper closing value and grasping delay;
- Add friction material to the gripper;
- Reduce the robot arm speed;
- Add grasp success detection logic, such as current detection or position feedback.

7.4 Coordinate Deviation Between Simulation and Real Robot

Phenomenon: Points that succeeded in simulation showed deviations on the real robot.

Cause: Assembly errors between the simulation model and the real robot, zero-point errors, and table calibration errors.

Solution:

- Re-calibrate the `base_link` and table coordinate frames;
- Correct pick point A and place point B by teaching;
- Configure real-robot coordinates separately in the parameter file without modifying the task logic.

7.5 Communication and Device Permission Issues

Phenomenon: Real-robot connection failed, or WiFi communication was unstable.

Cause: The computer was out of the EP WiFi hotspot range, or the network channel was congested.

Solution:

- Ensure the computer is within the EP WiFi hotspot range;
- Use STA mode (EP connected to a router) for more stable communication;
- Verify the SDK connection before starting the task.

7.6 Recovery After Emergency Stop

Phenomenon: After the emergency stop was triggered, the robot arm could not immediately continue the task.

Cause: In simulation, the active ROS2 Action goal is cancelled and the controller must be re-enabled. On the RoboMaster EP real robot, all ongoing arm and chassis actions are stopped via SDK calls.

Solution:

- **Simulation:** Reset the hardware, re-enable the controller, perform homing, and send the grasping Action again.
- **Real robot (EP):** All actions are stopped via SDK; manual homing (press R) is required before restarting the pick-place task.

7.7 ROS2 Action Timeout

This subsection describes timeout behaviour for ROS2-based simulation only. Timeout handling for the real-robot EP is implemented inside the Python SDK program. **Phenomenon:** The trajectory execution time was too long, and the Action timed out.

Cause: The speed scale was too low, there were too many trajectory points, or there was communication delay.

Solution:

- Appropriately increase the speed scale;
- Reduce unnecessary intermediate points;
- Increase the Action timeout;
- Use low-speed mode for the first validation, then gradually increase the speed.

7.8 RoboMaster EP SDK API Version Mismatch

Phenomenon: `TypeError: initialize() got an unexpected keyword argument` and `AttributeError: 'Robot' object has no attribute 'get_arm'` when connecting to the EP.

Cause: The installed `robomaster` SDK version differed from the documentation; API signatures and attribute names did not match.

Solution:

- Use `dir(robot)` to discover available methods and attributes at runtime;
- Access modules via direct attributes (`robot.robotic_arm`, `robot.gripper`) instead of getter methods;
- Use `conn_type="ap"` without passing the IP address in AP mode.

7.9 Chassis Rotation Cumulative Error

Phenomenon: When performing alternating left-right pick-and-place with chassis rotation, the chassis heading drifted progressively with each cycle.

Cause: The SDK `chassis.move(z=angle)` performs *relative* rotation, not absolute heading. Repeated calls with fixed angle values caused cumulative angular drift.

Solution:

- Maintain a `current_heading` state variable to track the current chassis orientation;
- Compute `delta = target_angle - current_heading` for each rotation;
- Update `current_heading` after each completed rotation.

7.10 Action Command Conflict

Phenomenon: Robot is already performing N action(s) error when sending arm and chassis commands in rapid succession.

Cause: The EP SDK does not allow concurrent actions on the same module; sending a new command before the previous one completes triggers a conflict.

Solution:

- Call `module.stop()` before each new action to clear pending commands;
- Add sufficient wait time between actions (`time.sleep()`);
- Use `action.wait_for_completed()` with a timeout to ensure each action finishes before the next.

8 Experiment Summary

8.1 Experimental Achievements

This experiment completed the simulation development and real-robot validation of fixed-point grasping with a robot arm. The main achievements are as follows:

1. A simulation scene containing the robot arm, gripper, table, and target object was built.
2. Fixed pick point A, fixed place point B, and a safe height were set.
3. The complete action flow was implemented: Homing → Move above pick point A → Descend → Grasp → Lift → Move above place point B → Descend → Release → Lift → Homing.

4. The robot arm was controlled through ROS2 nodes and Actions, and the robot arm status was published.
5. A single Launch file was used to start the simulation, control, and status nodes.
6. Unreachable targets, no IK solution, joint limit exceeded, collision, and emergency stop were detected and safely stopped.
7. In simulation, 5 consecutive grasps succeeded 5 times; on the real robot, 5 consecutive grasps succeeded 4 times, meeting the acceptance criteria.
8. The real-robot stage reused the simulation-stage task logic, replacing the ros2_control hardware interface with the RoboMaster EP Python SDK, modifying only device, communication, and position parameters without rewriting the task logic.
9. Each group member completed homing, movement, and safety stop operations and passed the safety operation check.

8.2 Experimental Takeaways

1. **ROS2 system integration capability:** Mastered the use of Topics, Services, Actions, and Launch files.
2. **Robot arm control capability:** Familiar with MoveIt2, ros2_control, joint trajectory control, and gripper control.
3. **Simulation-to-real migration capability:** Understood the differences between simulation and real robot and mastered the parameterized migration method, including cross-platform migration from ROS2-based simulation to SDK-based real-robot control.
4. **Exception handling capability:** Learned to safely handle unreachable targets, no IK solution, limit violations, collisions, and communication exceptions.
5. **Safety operation awareness:** Real-robot experiments must be run at low speed, with two-person cooperation and emergency stop priority, ensuring personnel and equipment safety.

8.3 Limitations and Future Improvements

1. **Positioning accuracy:** The real-robot placement had a deviation of about 5 mm, which can be improved by finer calibration and force compensation.
2. **Grasping reliability:** The gripper was not stable enough for smooth objects; force control or a different gripper can be added.
3. **Exception recovery:** Manual reset was required after an emergency stop; an automatic recovery process can be added.
4. **Vision assistance:** This experiment did not consider vision-based localization; a vision module can be added in the future for dynamic grasping.
5. **Trajectory optimization:** Trajectory speed, acceleration, and intermediate points can be optimized to reduce grasping time.
6. **Multi-target extension:** Multi-target grasping, target counting, and alarm functions can be added within the existing ROS2 framework.

8.4 Submission Materials

According to the experimental requirements, the submitted materials include:

1. ROS2 programs (simulation), Python SDK control programs (real robot), Launch files, and parameter files;
2. Simulation model, pick point, place point, and safe height configuration;
3. Simulation and real-robot demonstration videos;
4. 5 grasping result records;
5. Exception test records;
6. Brief experimental report;
7. Individual safety operation check records.

Appendix: Code Listings

```
1 pick_point:
2   x: 0.18
3   y: 0.00
4   z: 0.03
5   roll: 3.14
6   pitch: 0.00
7   yaw: 0.00
8
9 place_point:
10  x: 0.10
11  y: 0.15
12  z: 0.03
13  roll: 3.14
14  pitch: 0.00
15  yaw: 0.00
16
17 safe_height: 0.12
18 speed_scale: 0.1
19 gripper_open: 0.04
20 gripper_close: 0.02
```

Listing 2: Parameter configuration for pick and place points

```
1 IDLE
2   |
3   v
4 HOMING                Homing
5   |
6   v
7 MOVE_TO_A_ABOVE      Move above pick point A
8   |
9   v
10 DESCEND_A            Descend to pick point A
11  |
12  v
13 GRASP                Grasp the target object
14  |
```

```

15      v
16  LIFT_A                Lift to safe height
17      |
18      v
19  MOVE_TO_B_ABOVE      Move above place point B
20      |
21      v
22  DESCEND_B            Descend to place point B
23      |
24      v
25  RELEASE              Release the target object
26      |
27      v
28  LIFT_B                Lift to safe height
29      |
30      v
31  HOMING                Homing
32      |
33      v
34  DONE                  Task completed
35
36  In exceptional cases, enter:
37  SAFE_STOP             Safely stop or return to a safe position

```

Listing 3: State machine of the grasping task

```

1  ros2 launch arm_grasp_bringup grasp_sim.launch.py

```

Listing 4: Launch command

```

1  ros2 bag record /joint_states /tf /arm_status /task_result

```

Listing 5: Trajectory recording command

```

1  trial, pick_point, place_point, success, error_code, duration

```

Listing 6: Execution result format

```

1  E_UNREACHABLE
2  E_IK_FAIL
3  E_JOINT_LIMIT
4  E_COLLISION
5  E_TIMEOUT
6  E_ESTOP
7  E_GRASP_FAIL

```

Listing 7: Error codes

```

1  # RoboMaster EP connection
2  conn_type: "ap"           # WiFi AP mode
3  # No IP needed in AP mode (fixed 192.168.2.1)
4
5  # Arm coordinates (mm, origin = home position)
6  grasp_x: 155              # Grasp X: forward distance
7  grasp_y: -55              # Grasp Y: descend height
8  approach_height_mm: 50    # Safe approach height above grasp point
9  lift_mm: 70               # Lift height after grasping
10
11 # Gripper

```

```

12 open_power: 35                # Gripper open power
13 close_power: 50               # Gripper close power (can: medium)
14 gripper_seconds: 1.5          # Gripper action duration
15
16 # Chassis rotation
17 rotate_angle: 45               # Left/right alternation angle (degrees)
18 rotate_speed: 30               # Rotation speed (deg/s, range [10, 540])
19
20 # Arm movement limits
21 arm_x_range: [0, 190]          # X range (mm)
22 arm_y_range: [-90, 80]         # Y range (mm)
23
24 # Safety
25 timeout: 8.0                   # Action timeout (seconds)

```

Listing 8: RoboMaster EP real-robot parameter configuration

```

1 python ep_pick_place.py

```

Listing 9: Start real-robot control program

```

1 Detect exception
2   |
3   v
4 Stop current ongoing hardware action (arm.stop(), chassis.stop())
5   |
6   v
7 Cancel trajectory execution
8   |
9   v
10 Publish error (E_ESTOP / E_COLLISION / E_TIMEOUT)
11  |
12  v
13 Return to safe position or remain stopped
14  |
15  v
16 Wait for manual reset (press R to recenter)

```

Listing 10: Safety stop flow

```

1 [ACTION FAILED] close: state=action_idle, reason=grasp failed
2 [INFO] Returning to safe position...
3 [INFO] Safe stop completed.

```

Listing 11: Error message after failed grasp

```

1 ping 192.168.2.1

```

Listing 12: Verify WiFi connection to EP